

TEAM DETAILS

Project Name	AI FAQ Assistant API
College / University	Arulmigu Kapaleeswarar Arts and Science College
Department	B.Sc. Computer Science
Academic Year	2026 – 2027

Team Leader

Name	Usha. M
Roll No.	222411411
Email	sivvvaya@gmail.com

Team Members

S.No.	Name	Roll No.	Email
1	Kaviya	222411397	kaviyakaviramesh@gmail.com
2	Priya. P	2224111404	priya1625hsk@gmail.com
3	Priyanka. V	2224111405	Priyankavengatesanp@gmail.com
4	Rahman	2224111377	myidsabi06@gmail.com

AI FAQ Assistant API

AnswerBot – AI-powered FAQ and customer support automation

1 Project Description

The AI FAQ Assistant is a secure REST API for creating, managing and searching FAQs. It is built with Node.js and Express.js, stores data in MongoDB through Mongoose, protects routes with JWT authentication and bcrypt password hashing, and uses Google Gemini to generate questions, answers and topic suggestions automatically. Centralized error handling and schema validation keep the API reliable.

- JWT authentication and role-based access to protected routes
- AI-powered FAQ generation with Google Gemini
- Keyword search, category management and complete CRUD operations

2 Software Requirements

Software	Purpose
Operating System	Windows 10/11, macOS, or Linux (cross-platform support)
Node.js (v18 or above)	Runtime environment that runs the server logic and routing
npm (v9 or above)	Node package manager used to manage dependencies
Express.js (v4/v5)	Lightweight web framework used to build the RESTful endpoints
MongoDB / Mongoose ODM	NoSQL database storing users and FAQ data, with schema modelling

Postman / ThunderClient	API testing tools used to verify requests, responses and validation
Code Editor	Visual Studio Code or a similar IDE

3 Hardware Requirements

Component	Requirement
Processor	Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent
RAM	8 GB minimum (16 GB recommended for concurrent MongoDB instances)
Storage	1 GB of available disk space

4 Project Architecture

The application exposes structured endpoints handled by a layered server architecture. Incoming requests pass through middleware filters before they reach the services and the database.

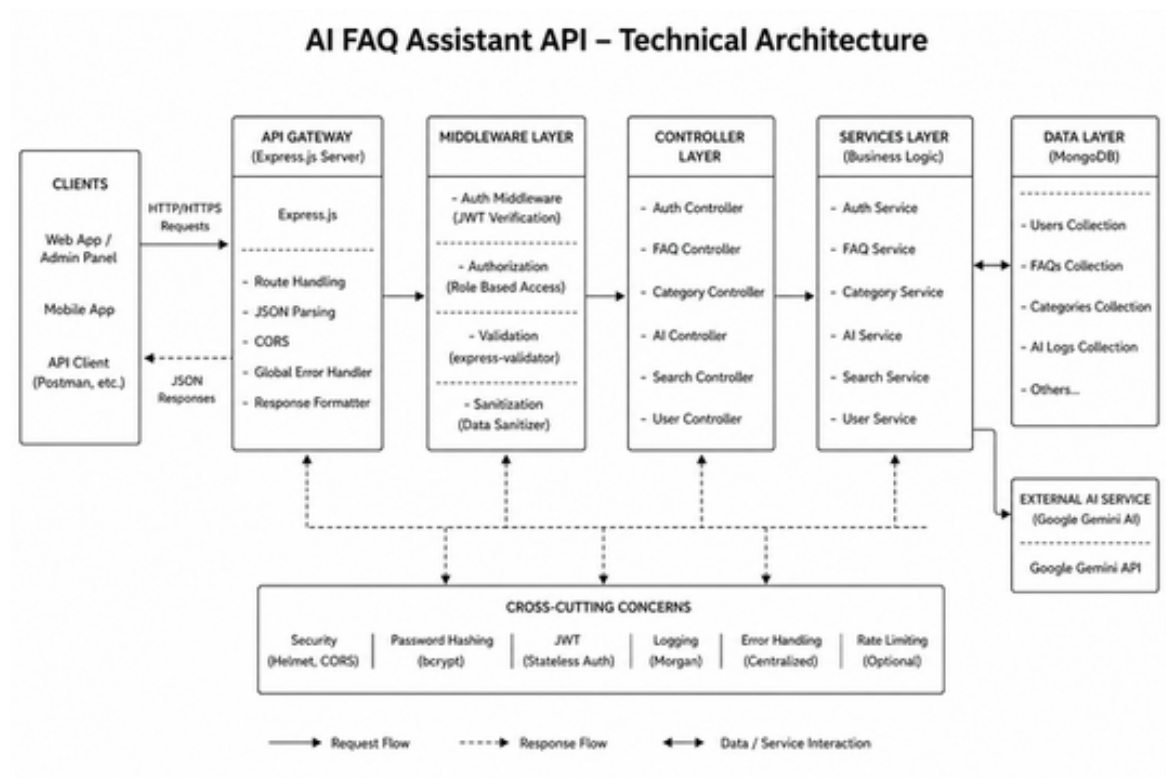


Figure 1: AI FAQ Assistant API – Technical Architecture

Components Definition

Component	Description
Interface	API tools like Postman query distinct route paths to extract JSON entities.
Express Server Gateway	Monitors network ports, maps HTTP methods (GET, POST, PUT, DELETE), parses JSON payloads and handles CORS

	headers.
Authentication Middleware	Extracts tokens from headers, validates user context signatures and checks authorization for guarded routes.
Granular Services	Logic modules that enforce domain rules for user sessions and query operations.
Database Interface Layer	Mongoose translates application logic into performant native MongoDB queries.
AI Flow (Service Integration)	Connects to Google Gemini through its SDK so the controller can offload question-answering and automated topic generation.

5 Roles and Responsibilities

● Manage all registered users ● View, update and delete any FAQ ● Monitor AI-generated content ● Manage FAQ categories ● Access complete system functionality	● Create new FAQs ● Edit personal FAQ records ● Delete owned FAQs ● Generate FAQs using AI ● Organize FAQs under categories
● View public FAQs ● Search FAQs using keywords ● Generate AI-assisted answers ● Access personal profile information	● View published FAQs ● Search FAQs ● Cannot create, edit or delete FAQs ● No access to protected endpoints

6 User Flow Diagram

A user registers or logs in, receives a JWT token, and is directed by role (Admin, Editor, Author or Reader). Every authenticated request follows the same path: token validation, role-based authorization, action, then JSON response.

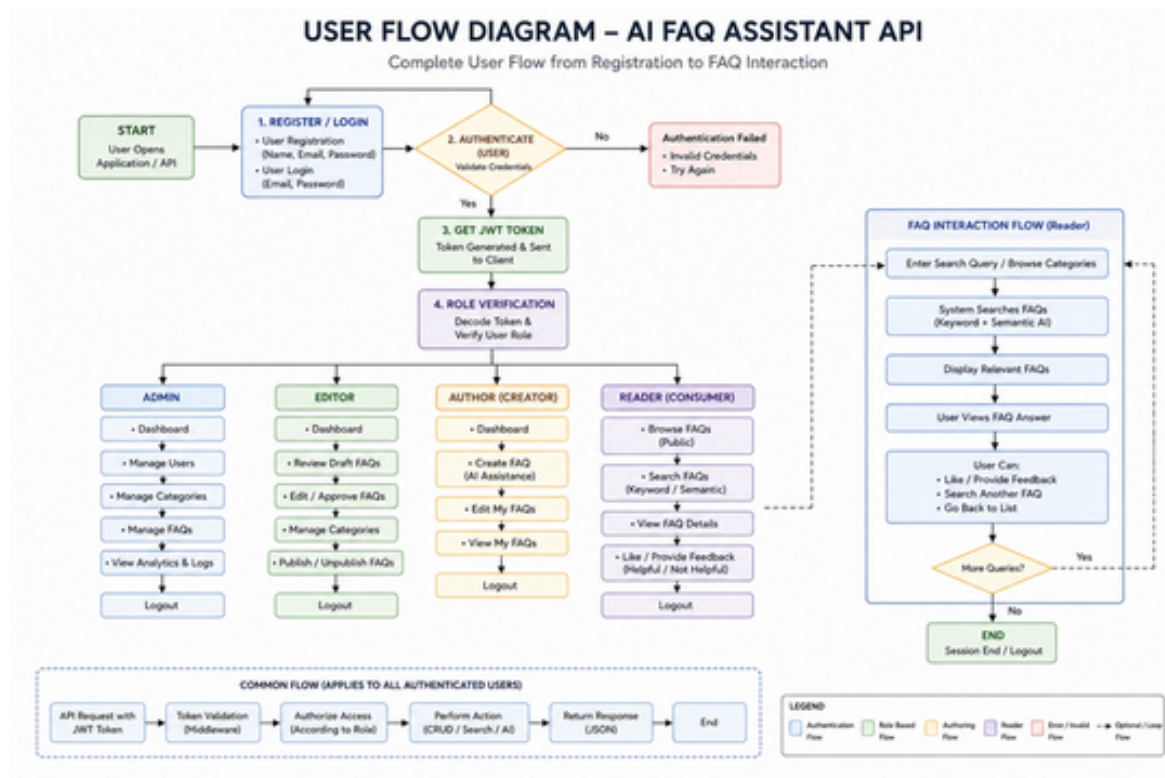


Figure 2: User Flow Diagram – AI FAQ Assistant API

7 Project Setup and Configuration

Creating the Project Folder

1. Create a new folder with your project name.
2. Inside it, create a subfolder named src.
3. Open the folder in VS Code.

Server Setup (npm init)

Open the root folder terminal in VS Code and run:

```
npm init -y  
  
npm install express mongoose dotenv cors jsonwebtoken bcrypt @google/genai
```

File and Folder Structure

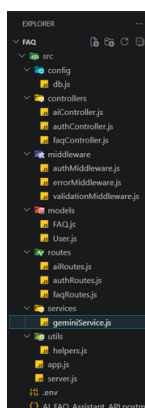


Figure 3: Project folder structure

8 Project Execution and API Testing Demo

Install Postman or ThunderClient (VS Code extension), then start the backend server:

```
npm run dev
```

Endpoint	Method	Route	Access
1. Register	POST	/api/auth/register	Public
2. Login	POST	/api/auth/login	Public
3. Create FAQ	POST	/api/faqs	Private (Bearer JWT)
4. Search FAQs	GET	/api/faqs/search?q=configure	Public
5. Generate FAQ (AI)	POST	/api/ai/generate-faq	Private (Bearer JWT)

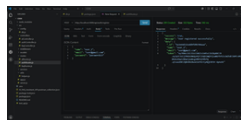
Sample Request Bodies

```
// POST /api/auth/register
{ "name": "Priya Sharma", "email": "priya@writeflow.com", "password": "securepassword123" }

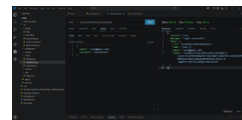
// POST /api/faqs
{ "question": "How do I configure custom category tags?",
  "answer": "Navigate to settings panel, append keyword tags arrays, and save
```

```
schema.",  
  "category": "Configuration" }  
// POST /api/ai/generate-faq  
{ "topic": "Mongoose schema indexing validation runtime workflow optimization" }
```

Test Results



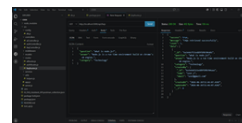
Register – 201 Created



Login – 200 OK



Create FAQ – 201 Created



Search FAQs – 200 OK

API Testing Video: https://drive.google.com/file/d/1u_w1sigiQynhmqTDbj4NBm3CLIMunEWN/view?usp=sharing

Demo Link: https://drive.google.com/file/d/1ajAZGgOaowPdyW2x5otcsLI84NeomSYJ/view?usp=drive_link